

TP — Adapter un script d'évaluation CamemBERT

 Durée : ~30 minutes

 Objectif : Comprendre et modifier un script d'inférence en classification binaire

Contexte

Vous disposez d'un modèle **CamemBERT fine-tuné** pour classifier des mails en :

- **Answer**
- **NoAnswer**

Le script fourni permet d'évaluer ce modèle sur un fichier CSV contenant :

```
mail;label
```

Exemple :

```
mail;label
"Pouvez-vous me rappeler ?";Answer
"Newsletter promotionnelle";NoAnswer
```

Objectifs du TP

À la fin du TP, vous devez être capable de :

- Comprendre les différentes parties d'un script Transformers
- Comprendre la chaîne :

```
texte → tokenizer → modèle → logits → softmax → prédiction
```

- Implémenter correctement un mapping texte ↔ entier

Partie 1 — Lecture du script

Travail demandé

Dans le script fourni, identifiez et commentez clairement les 5 blocs suivants :

1. Chargement des arguments
2. Chargement du modèle et du tokenizer
3. Lecture et préparation du CSV
4. Boucle d'inférence (tokenization + modèle + softmax)
5. Calcul des métriques (accuracy + matrice de confusion)

Ajoutez des commentaires comme ceci :

```
# =====  
# 1. Chargement des arguments  
# =====
```

Partie 2 — Implémenter le mapping des labels

Dans le script, les deux fonctions suivantes sont volontairement incomplètes :

```
def label_to_int(x: str) -> int:  
    pass  
  
def int_to_label(i: int) -> str:  
    pass
```

Travail demandé

Complétez ces deux fonctions pour que :

- "Answer" → 1
- "NoAnswer" → 0

 Contraintes :

- Supprimer les espaces inutiles
- Lever une erreur si le label est inconnu
- Le script doit fonctionner sans modifier le reste du code

Question (à répondre par écrit)

Pourquoi le modèle travaille-t-il avec des entiers (0/1) et pas directement avec du texte ?

Partie 3 — Test du script

On veut maintenant pouvoir tester le modèle sans CSV.

Nouvelle fonctionnalité

Ajouter un argument :

```
parser.add_argument("--text", help="Phrase unique à classifier")
```

Et modifier le `main()` pour que :

- Si `--text` est fourni → le script ne lit pas le CSV
- Il affiche directement :
 - la prédiction
 - le score

Partie 4 — Remplacer l'Accuracy par le Recall (10 min)

Contexte

Actuellement, le script calcule :

```
acc = correct / total if total else 0.0  
print(f"Accuracy: {acc:.3f}")
```

Mais dans un système de gestion de mails professionnels, l'accuracy n'est pas forcément la meilleure métrique.

Si votre dataset contient beaucoup plus de **NoAnswer** que de **Answer**, un modèle peut avoir une très bonne accuracy... tout en ratant beaucoup de mails importants.

Travail demandé

1. Supprimez l'affichage de l'Accuracy.
2. Ajoutez le calcul du **Recall pour la classe Answer**.
3. Affichez : `Recall (Answer): 0.xxx`

 Attention à la division par zéro.



Partie 5 - Mettre le contenu sur GitHub

Mettez votre script modifié et commenté (partie 1) sur GitHub ainsi que la copie d'écran du résultat de whoami, du script exécuté avec une seule phrase (partie 3) et du script exécuté avec la modification du Recall (partie 4)